COL1101 Lecture 21 2026/09/10

Topic: Fast Buildheap, HeapSort

Announcements

Quiz 2 September 19 08:30 – 09:30

Seating arrangement on exam.iitd.ac.in

Syllabus: COL1101 Lectures 1-21 + COL1000 + JEE

Next class? Options: September 20 18:30-19:30

Recap

Heap Operations:

- HeapUp
- HeapDown
- Enqueue
- ExtractMin
- ChangeKey

} $O(\log n)$ each

- BuildHeap "Obvious algorithm": $\Theta(n \log n)$

Build Heap (l) l: list of n keys

Construct a heap over the set of keys in list l.

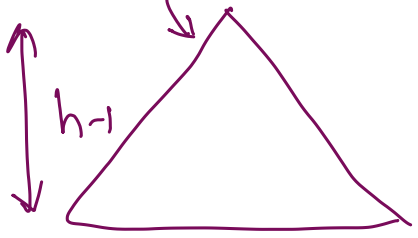
Obvious algorithm: Start with an empty heap; enqueue each element of l.

Running time : $O(n \log n)$

$l = [n, n-1, \ldots, \overbrace{2^{h-1}}^{2^{h-1}-1}, \overbrace{2^{h-2}, \ldots, 1}^{\approx \frac{n}{2}}]$

$n = 2^h - 1$

$h \approx \log_2 n$



$\Omega(h)$ insertion time for each of these

$\therefore$ The "Obvious algorithm" takes $\Theta(n \log n)$ time.

Recall HeapDown (u)

Preconditions :

① Almost complete binary tree

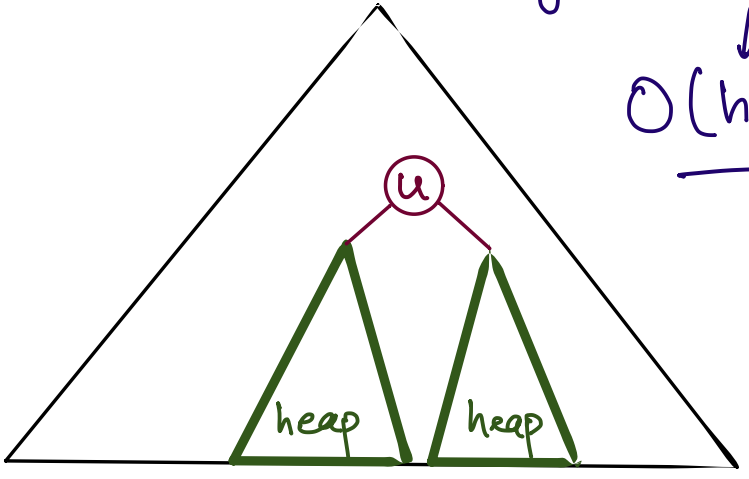
② Left and right subtrees of u are both heaps.

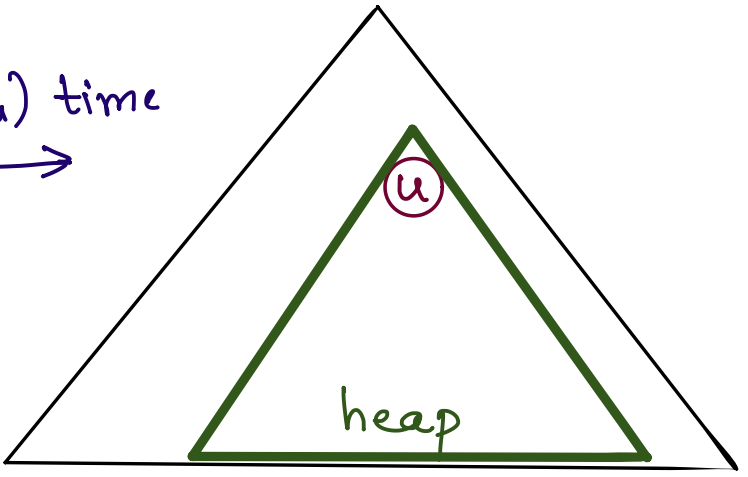
Postcondition : Subtree rooted at u is a heap.

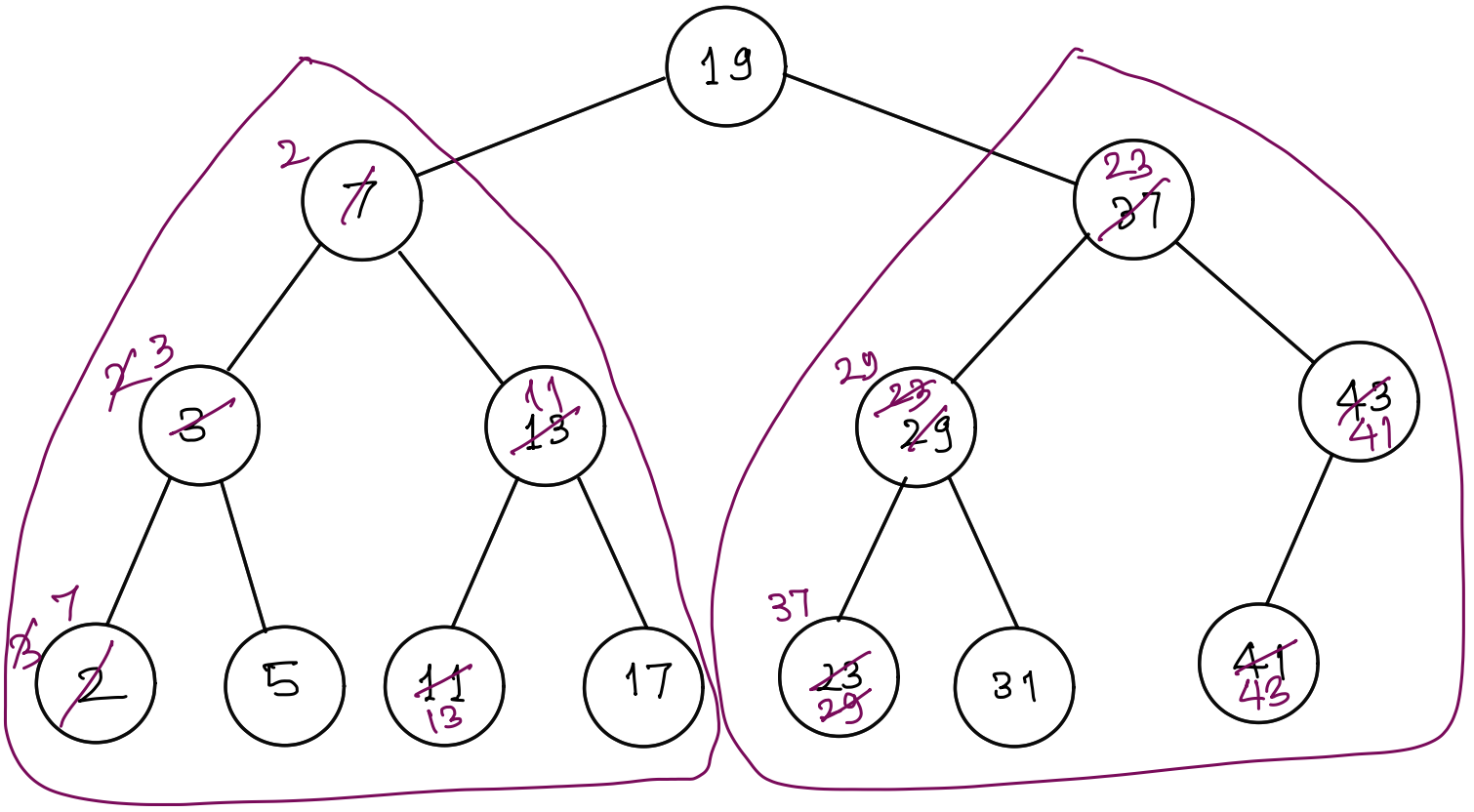
height of subtree rooted at u

$\downarrow$

$O(h_u)$ time







Fast BuildHeap (l)

For u = len(l) − 1 to 0 :

Loop invariant: $\forall v \in \{u+1, \ldots, len(l)-1\}$ subtree rooted at v is a heap.

HeapDown (u).

$\forall v \in \{u, \ldots, len(l)-1\}$ subtree rooted at v is a heap.

Upon termination : u = −1

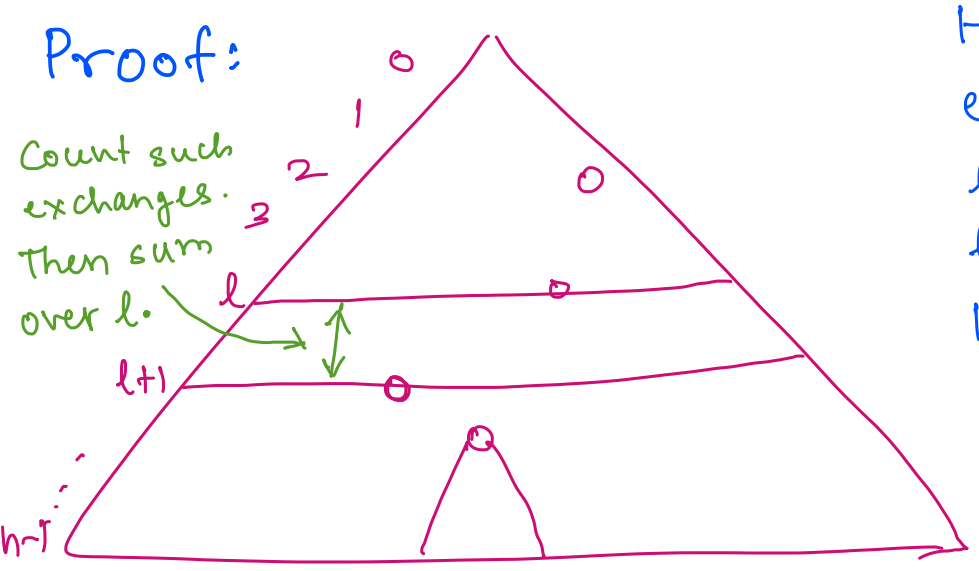
∴ Loop invariant ⇒ Subtree rooted at 0 is a heap.

i.e. the entire tree is a heap.

Time Complexity of Fast BuildHeap

Claim: FastBuildHeap(l) runs in $O(n)$ time, where $n = \text{len}(l)$.

Proof:



Heapdown(u) causes 0 exchanges between levels $l, l+1$ if u lies on level $l+1$ or below.

Heapdown(u) causes ≤ 1 exchanges between levels $l, l+1$ if u lies on level l or above.

$\therefore$ Total # exchanges between levels l $l+1$ $\leq$ # nodes on levels $0 \cdots l$ $= 2^{l+1} - 1 \leq 2^{l+1}$

$\therefore$ Total # exchanges $\leq \sum_{l=0}^{h-2} 2^{l+1} \leq 2^h$, which $O(n)$ because $h \approx \log_2 n$.

Heap Operations : Summary

- HeapUp(u) — $O(\text{depth}(u))$
  - Enqueue — $O(\log n)$
- HeapDown — $O(h_u) = O(\log n - \text{depth}(u))$
  - ExtractMin — $O(\log n)$
  - ChangeKey — $O(\log n)$
  - BuildHeap — $O(n)$

Find Min, Size, Is Empty — $O(1)$

Sorting Algorithm	Worst-case Time Complexity	Worst-Case Extra Space
Bubble, Selection, Insertion	$\Theta(n^2)$	$\Theta(1)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n)$
Quicksort	$\Theta(n^2)$ *	$O(1)$

(*: $\Theta(n \log n)$ time if median picked as pivot.)

Fact: Can find median in $O(n)$ time.

(Ask your COL2051/MTL2042 instructor how.)

Yet Another Sorting Algorithm : Heapsort.

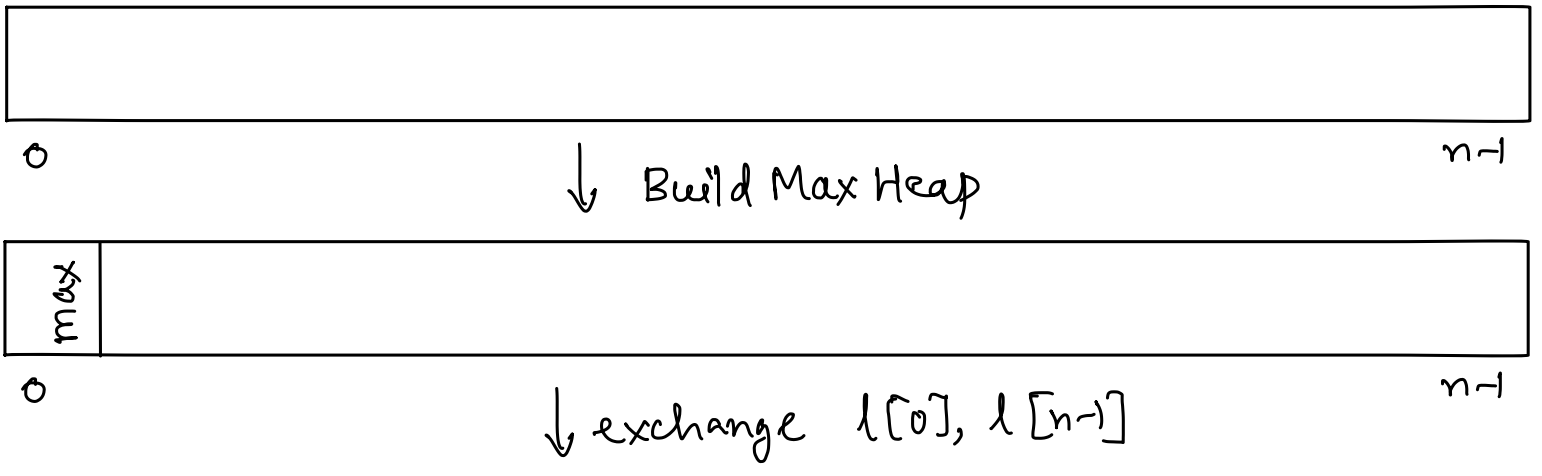
HeapSort (A):

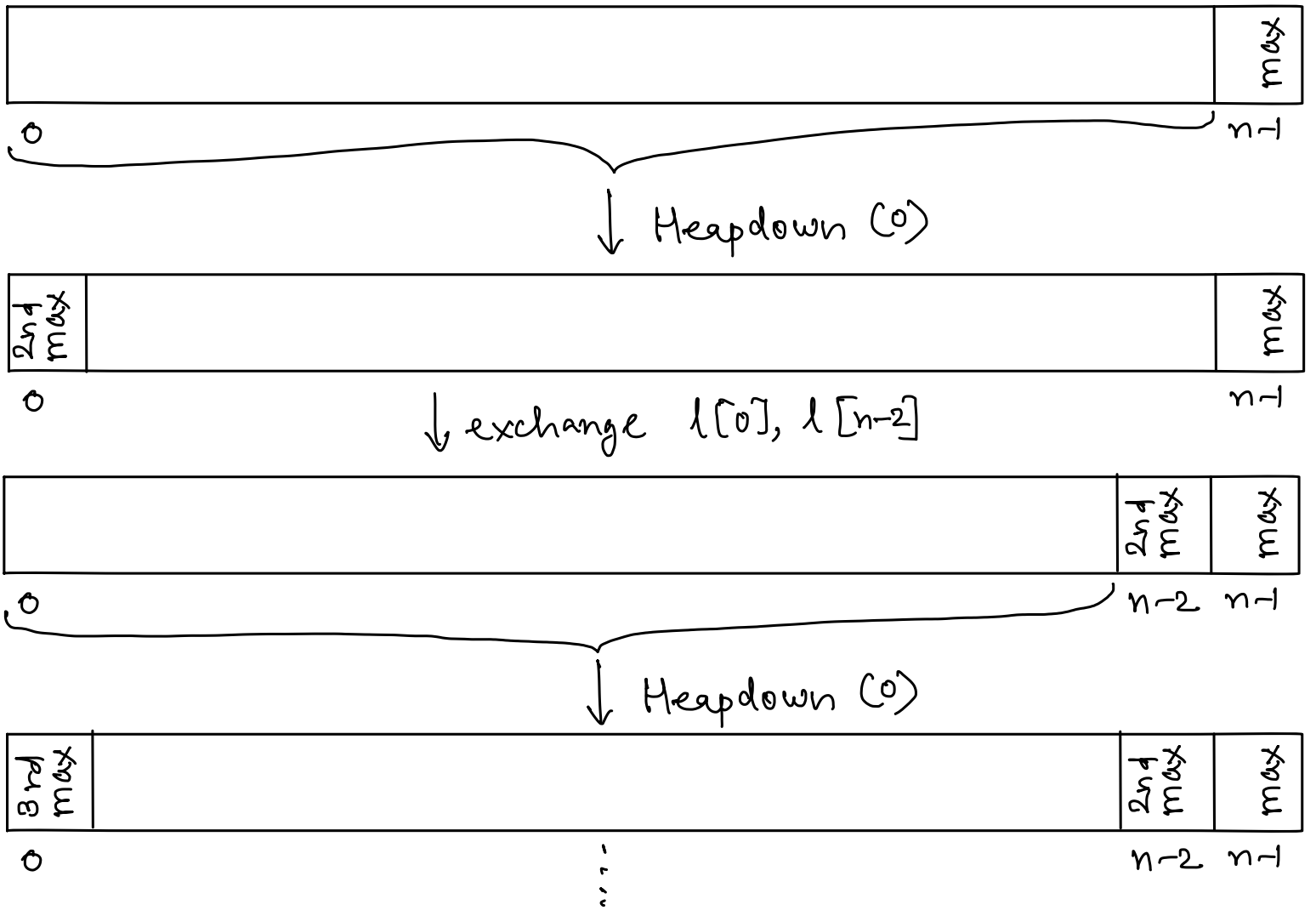
H ← Build Heap (A)

Repeatedly H.ExtractMin() and report the answer.

Running Time : $O(n \log n)$

In-place HeapSort





Claim: HeapSort can be performed in $O(1)$ extra space (and $O(n \log n)$ time).

In-place Heapsort (l):

- Make l a max-heap in-place
- For i = size(l) − 1 to 0:

  (Invariant: l[0:i+1] is a max heap; l[i+1:] contains the size(l) − i − 1 largest elements of l in non-decreasing order.)

  Exchange l[0] and l[i]

  Make l[0, ..., i−1] a heap using HeapDown.

Time: $O(n \log n)$ Extra space: $O(1)$.